## Goal

The goal of this lab session is to gain insight in how different thread blocks are divided into warps before they are scheduled on the GPU. We compare different number of threads per warp and the effect on performance. Additionally, we compare how we can further optimize our code by taking into account the execution time difference of different threads.

## Warps

Before we can assess the performance impact of optimizing for thread warps, we should first explain what thread warps are. In the previous lab, we explained that to execute multiple instances of a kernel function, a grid of thread blocks is created. We assumed that each thread block contained x amount of threads. The threads in the block are now divided into groups of 32 threads called warps. Each warp should execute the same kernel function and be assigned to the same thread block. How a function is scheduled and executed greatly depends on the amount of warps used. For example, if 32 warps with 1 thread are used 1024 cores will be designated wile only 32 threads are used. The reason is that only warps of 32 threads can be scheduled on the GPU. If we were to create one warp of 32 threads, we get the same amount of threads while only using 32 cores. In the lab we performed a test to illustrate the effect of inefficient warp utilization. We convert an image to an array of RGB colors, then the colors are inverted where each thread calculates one element inversion. The program was run using different thread per blocks to illustrate the effect warps have on the overall time.

1. 62106 blocks of 32 threads → 1.694304 milliseconds to invert
2. 60224 blocks of 33 threads → 1.950496 milliseconds to invert
3. 10406 blocks of 191 threads → 2.777440 milliseconds to invert
4. 10351 blocks of 192 threads → 0.428800 milliseconds to invert
5. 10298 blocks of 193 threads → 0.462208 milliseconds to invert

Looking at these numbers we see that for blocks of 32 threads the inversion time is faster than for 33. This can be explained since the 33 threads per block will require 2 warps to be scheduled on the processor, while only 1 warp is needed for 32 threads per block. For 192 threads per block we can note the same results. If the number of threads per block is divisible by 32 we can assume the execution time is lower since a whole warp is used and all the allocated cores are being utilized.

## Thread divergence

In the previous section about thread warps, we assumed that each thread in a thread warp executed the same function. If within a thread warp not all the threads were to have the same execution time some threads would be finished executing while others are not this is called thread divergence. Continuing on the previous example we used an array of RGBRGB colors to represent an image. We now created a longer calculation on the R part of the array that will cause in increase in execution time on some threads and by extension the entire warp. below is an example with and without longer calculation for the R color.

1. 5175 blocks of 192 threads 0.770688 milliseconds without longer calculation
2. 5175 blocks of 192 threads 0.914688 milliseconds with longer calculation

We can see that increasing the calculation on R the overall execution time significantly increases. This is due to the thread divergence within a warp. We can see that the number of blocks has decreased compared to the first example where the same image was used. This is achieved by using striding, a thread will now execute multiple elements of the image array if there are not enough threads created to cover the whole image array.

## Conclusion

On the GPU a minimum of one warp of 32 threads is scheduled and executed on the GPU. When programming and using this knowledge we illustrated that the execution time can be decreased.